CISC 886 — Cloud Computing

School of Computing, Queen's University, Kingston, Canada

PROJECT REPORT

Cloud-Based Conversational Chatbot

Fine-Tuning Qwen2.5-1.5B-Instruct on StackOverflow Python Q&A

Student NetID	25jdvr
Course	CISC 886 — Cloud Computing
Institution	Queen's University, Kingston, Canada
Submission Date	April / May 2026
Base Model	Qwen2.5-1.5B-Instruct (Unsloth)
Dataset	StackOverflow Posts — Python Q&A (~600K pairs)
AWS Region	us-east-1 (N. Virginia)
S3 Bucket	25jdvr-chatbot-bucket

Section 1 — System Architecture

The system implements a complete, end-to-end MLOps pipeline for a Tech Support chatbot fine-tuned on StackOverflow data and deployed on AWS. All infrastructure is provisioned via Terraform under the project identifier 25jdvr. The architecture is organised into five functional layers: data ingestion, distributed preprocessing, model fine-tuning, model serving, and user interaction.

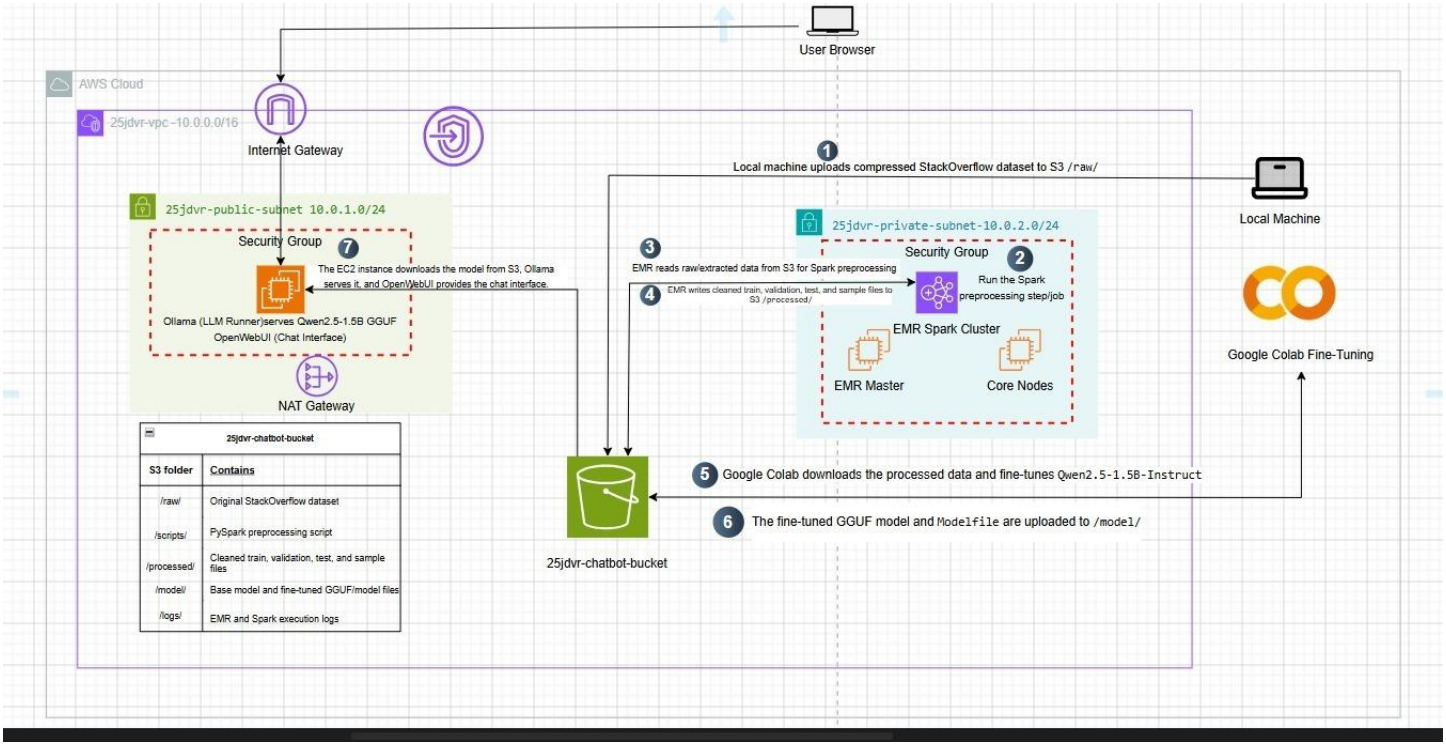

Figure 1.1 — End-to-end system architecture deployed on AWS (drawn in draw.io). The diagram shows the VPC with public and private subnets, EMR cluster, EC2 instance, S3 bucket, Internet Gateway, NAT Gateway, and Google Colab fine-tuning step.

The architecture diagram (Figure 1.1) shows all seven numbered steps of the data flow, which proceed as follows. In Step 1, a local machine uploads the compressed StackOverflow dataset (stackoverflow-Posts.7z, 18.6 GB) to the S3 bucket 25jdvr-chatbot-bucket under the /raw/ prefix. In Step 2, an EMR Spark cluster (25jdvr-spark-cluster) is launched inside the private subnet and a PySpark preprocessing job is submitted as an EMR Step. In Step 3, the Spark job reads raw XML data from S3 /raw/. In Step 4, EMR writes the cleaned, tokenised, and split output files (train, validation, test, and sample) back to S3 /processed/. In Step 5, Google Colab downloads the processed data from S3 and fine-tunes the Qwen2.5-1.5B-Instruct model using QLoRA via the Unsloth library. In Step 6, the resulting GGUF model file and its Modelfile are uploaded to S3 /model/. Finally, in Step 7, the EC2 instance in the public subnet downloads the fine-tuned model from S3, Ollama serves it locally, and OpenWebUI provides a browser-accessible chat interface on port 3000.

The S3 bucket 25jdvr-chatbot-bucket acts as the central data store across all pipeline stages. Its folder structure is: /raw/ for the original compressed dataset; /scripts/ for the bootstrap shell script; /processed/ for the cleaned Parquet splits; /model/ for the base and fine-tuned GGUF model files; and /logs/ for EMR and Spark execution logs.

Section 2 — VPC & Networking

2.1 Infrastructure-as-Code Choice

All networking resources were provisioned using Terraform, not the AWS Console. Terraform was chosen because it produces reproducible, version-controlled infrastructure, allows the entire 30-resource stack to be created or destroyed with a single command, and eliminates the risk of manual configuration drift. The Terraform state captures every resource parameter, making the deployment fully auditable. The execution plan shown in Figure 2.1 documents every resource that Terraform planned to create before apply, and Figure 2.2 confirms that all 30 resources were created successfully.

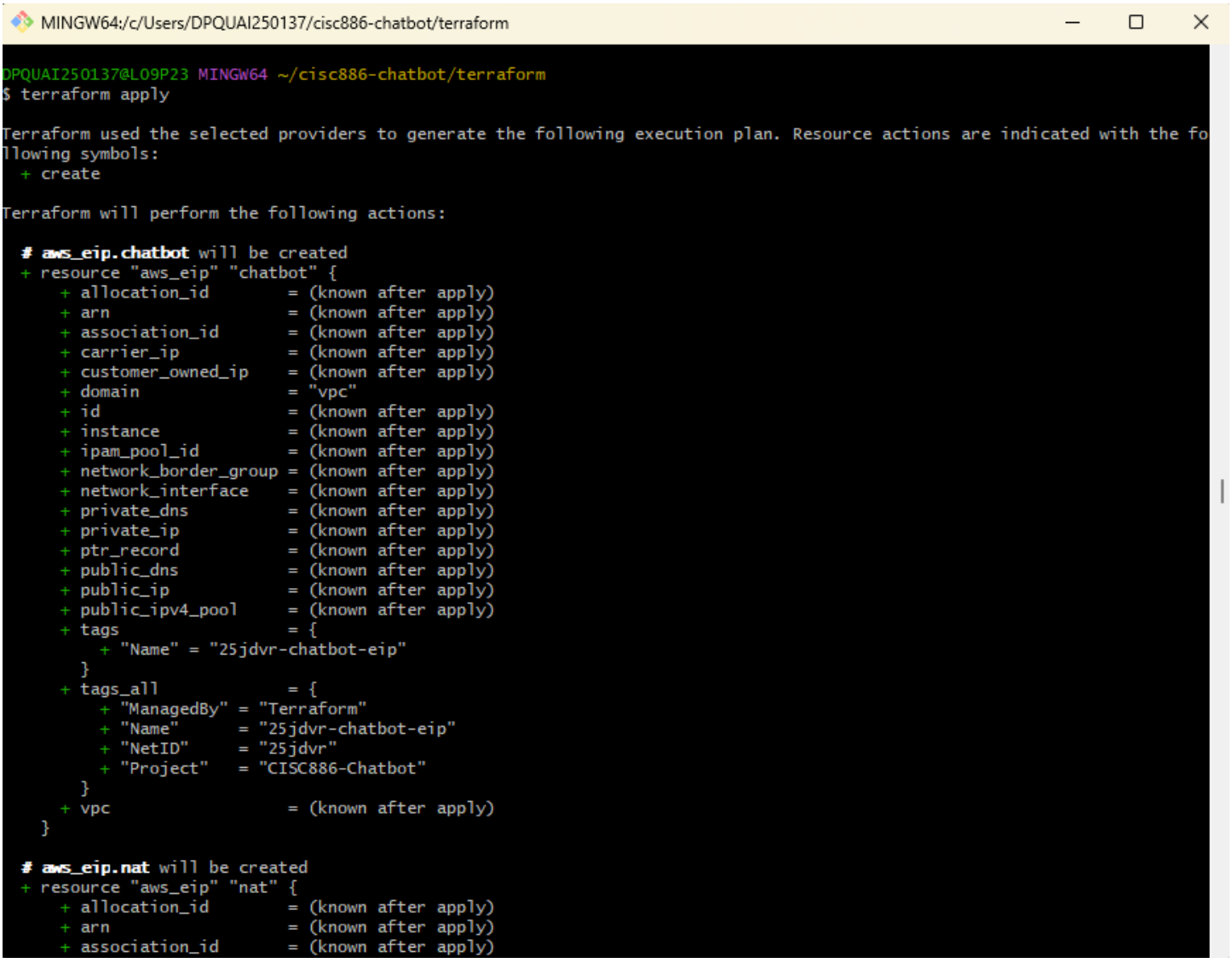

Figure 2.1 — terraform apply execution plan showing the first resources to be created, tagged with the project NetID 25jdvr and ManagedBy=Terraform.

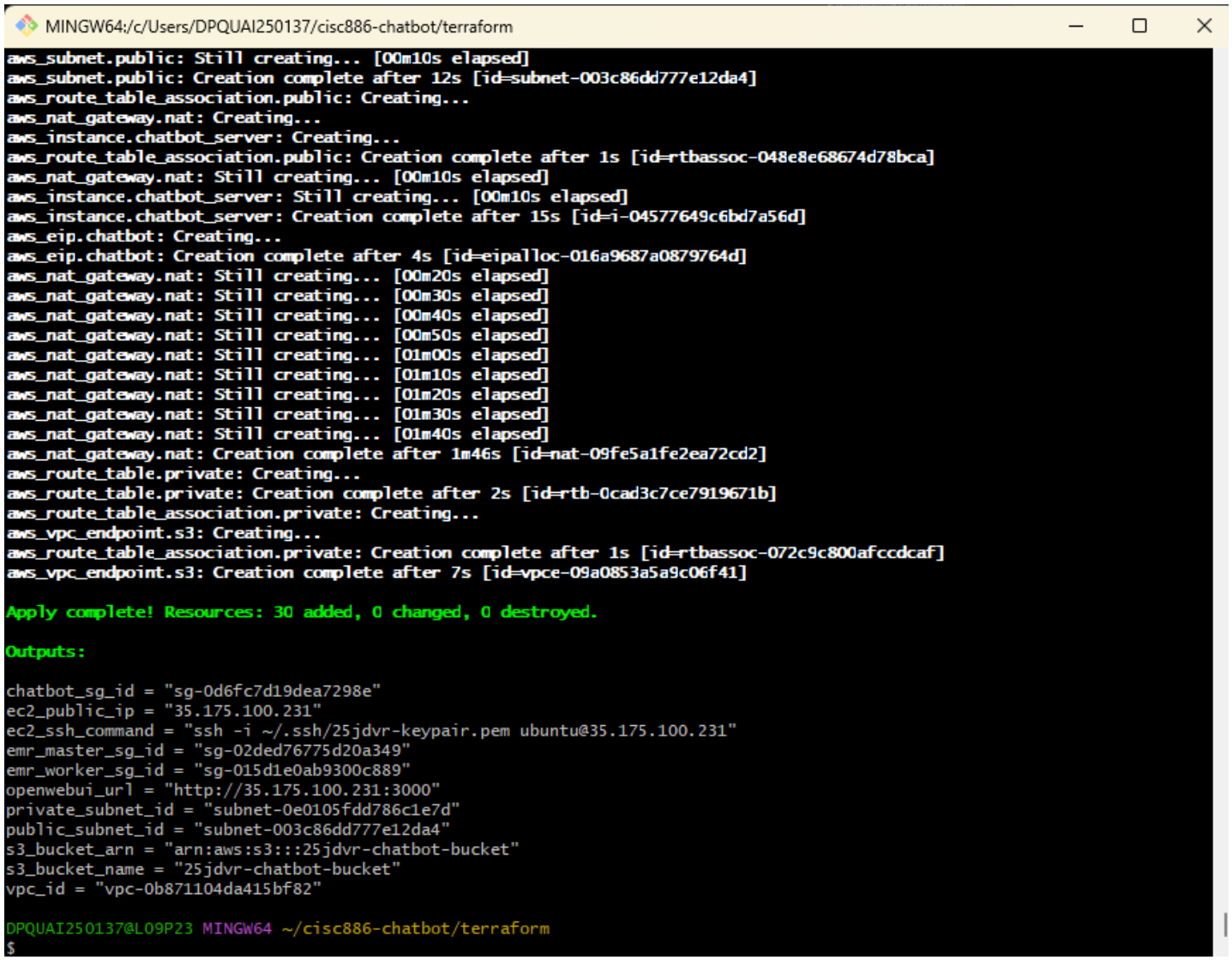

Figure 2.2 — Successful terraform apply output. 30 AWS resources were created. Key outputs include: VPC ID vpc-0b871104da415bf82, public subnet subnet-003c86dd777e12da4, EC2 public IP 35.175.100.231, and OpenWebUI URL http://35.175.100.231:3000.

2.2 VPC and CIDR Design

A dedicated VPC named **25jdvr-vpc** was created with CIDR block **10.0.0.0/16**, providing 65,536 available IP addresses. The /16 block was chosen to give sufficient address space for both subnets and future expansion. The VPC is split into two subnets across the same Availability Zone in us-east-1.

Subnet	CIDR Block	Type	Purpose	Route
25jdvr-public-subnet	10.0.1.0/24	Public	EC2 (Ollama + OpenWebUI)	Internet Gateway
25jdvr-private-subnet	10.0.2.0/24	Private	EMR Spark Cluster	NAT Gateway + S3 VPC Endpoint

Table 2.1 — Subnet design summary for the 25jdvr VPC.

The public subnet (10.0.1.0/24) hosts the EC2 instance that needs to be reachable by users. It is connected to the Internet Gateway so inbound HTTP traffic on port 3000 (OpenWebUI) and SSH on port 22 can reach the instance. The private subnet (10.0.2.0/24) hosts the EMR cluster. EMR nodes are placed in the private subnet because they do not require direct internet exposure — they communicate only with S3 (via a VPC Endpoint, keeping traffic off the public internet) and with each other. The NAT Gateway in the public subnet allows EMR nodes to initiate outbound internet connections when needed (e.g., to download packages during bootstrap) without exposing them to inbound traffic.

2.3 Security Group Rules

Two security groups were configured: one for the EC2 chatbot instance and one for the EMR cluster.

EC2 Chatbot Security Group (25jdvr-chatbot-sg)

Port	Protocol	Source	Justification
22	TCP	0.0.0.0/0	SSH access for administration and model loading
3000	TCP	0.0.0.0/0	OpenWebUI browser interface (user-facing)
11434	TCP	VPC CIDR	Ollama API — internal only, not exposed publicly
All	All	0.0.0.0/0 (egress)	Allow outbound for S3 model download and package installs

Table 2.2 — EC2 security group rules.

EMR Security Groups (25jdvr-emr-master-sg / 25jdvr-emr-worker-sg)

The EMR master and worker security groups follow AWS-managed EMR rules. Intra-cluster communication (master-to-worker and worker-to-worker) is allowed on all ports within the private subnet CIDR. No public inbound rules are applied. All S3 access is routed through the VPC Gateway Endpoint for S3, avoiding internet egress costs and exposure. The separation of master and worker security groups follows the principle of least privilege: only the master node needs the resource manager port (8088/YARN) accessible from the VPC, while workers only communicate with the master.

2.4 Gateways and Routing

- Internet Gateway (25jdvr-igw): Attached to the VPC and associated with the public subnet route table. Enables the EC2 instance to be reachable from the internet and to download packages.
- NAT Gateway (25jdvr-nat): Deployed in the public subnet with an Elastic IP (25jdvr-chatbot-eip). The private subnet route table points 0.0.0.0/0 to this NAT Gateway, allowing EMR nodes to initiate outbound connections.
- S3 VPC Gateway Endpoint: Added to the private subnet route table so EMR-to-S3 traffic stays within the AWS network. This eliminates data transfer charges for S3 access from EMR and improves throughput for the 94 GB dataset read.
- Elastic IP (25jdvr-chatbot-eip): Assigned to the EC2 instance (35.175.100.231) to provide a stable public address that persists across reboots, ensuring the OpenWebUI URL does not change.

Section 3 — Model & Dataset Selection

3.1 Model Selection

Property	Value
Model name	Qwen2.5-1.5B-Instruct
Parameter count	1.5 billion
Source	Hugging Face / Unsloth (unsloth/Qwen2.5-1.5B-Instruct-bnb-4bit)
Format used	GGUF (exported after QLoRA fine-tuning via Unsloth)
License	Apache 2.0 (permissive, allows commercial use and modification)
Quantisation	4-bit (BnB NF4) for training; Q4_K_M GGUF for inference

Table 3.1 — Model selection details.

Qwen2.5-1.5B-Instruct was selected for three reasons. First, at 1.5 billion parameters it fits comfortably within the VRAM budget of a free Google Colab T4 GPU (15 GB) when combined with 4-bit quantisation, allowing fine-tuning to complete without out-of-memory errors. Second, Qwen2.5 is an instruction-tuned model, meaning it is already trained to follow question-and-answer formatting, which aligns directly with the StackOverflow Q&A structure used in this project. Third, the Unsloth library provides a pre-quantised version of this model (unsloth/Qwen2.5-1.5B-Instruct-bnb-4bit), making it straightforward to load, fine-tune with QLoRA, and export to GGUF format for serving with Ollama. The Apache 2.0 licence imposes no restrictions on academic or research use.

3.2 Dataset Selection

Property	Value
Dataset name	StackOverflow Posts (Posts.xml)
Source	Stack Exchange Data Dump (https://archive.org/details/stackexchange)
Raw compressed size	18.6 GB (stackoverflow-Posts.7z)
Raw extracted size	94.0 GB (Posts.xml)
Record type	Question and accepted-answer pairs (XML)
Domain	Tech Support / Programming Q&A
License	Creative Commons Attribution-ShareAlike 4.0 (CC BY-SA 4.0)

Table 3.2 — Dataset details.

The StackOverflow Posts dataset was selected because it provides millions of real programming Q&A pairs, directly matching the Tech Support chatbot use case. The dataset's Creative Commons licence permits academic use. The raw XML was uploaded to S3 /raw/ before any processing was performed, ensuring that the original data is preserved and reproducible.

3.3 Train / Validation / Test Split Strategy

After Spark preprocessing, the cleaned Q&A pairs were written to three stratified Parquet splits. The split sizes and rationale are described in Section 4. Data leakage was avoided by performing the split after all text cleaning and deduplication steps, using a deterministic random seed, so no row from any split could appear in another split. The Spark job never reads from the output prefix during preprocessing, eliminating any feedback loop.

3.4 Sample Q&A Pair

The following is a representative sample verbatim from the processed dataset, showing the ChatML format used for fine-tuning:

```
<|im_start|>user
Write a Python function that returns the largest number in a list.<|im_end|>
<|im_start|>assistant
def find_max(nums):
    if len(nums) == 0:
        return None
    max_num = nums[0]
    for num in nums:
        if num > max_num:
            max_num = num
    return max_num<|im_end|>
```

Sample 3.1 — A verbatim instruction/response pair from the processed dataset formatted in ChatML, as displayed in the deployed OpenWebUI interface.

Section 4 — Data Preprocessing with Apache Spark on EMR

4.1 EMR Cluster Configuration

Parameter	Value
Cluster name	25jdvr-spark-cluster
EMR release	emr-6.15.0
Spark version	3.4.1
Region	us-east-1 (N. Virginia)
Primary node	1 x m5.xlarge (4 vCPU, 16 GiB RAM)
Core nodes	2 x m5.xlarge (4 vCPU, 16 GiB RAM each)
Total cluster RAM	48 GiB (3 nodes × 16 GiB)
EBS root volume	15 GiB, gp3, 3000 IOPS, 125 MiB/s per node
Instance group type	Uniform (same instance type for primary and core)
Log destination	S3 (aws-logs-507519853509-us-east-1/elasticmapreduce)
VPC placement	25jdvr-private-subnet (10.0.2.0/24)

Table 4.1 — EMR cluster configuration.

The m5.xlarge instance type was selected because it provides sufficient memory (16 GiB per node) to hold Spark executor JVMs and shuffle data while processing the 94 GB XML file in parallel. Using the private subnet ensures that the cluster is not exposed to the internet while still having S3 access via the VPC Endpoint.

4.2 EMR Configuration Screenshots

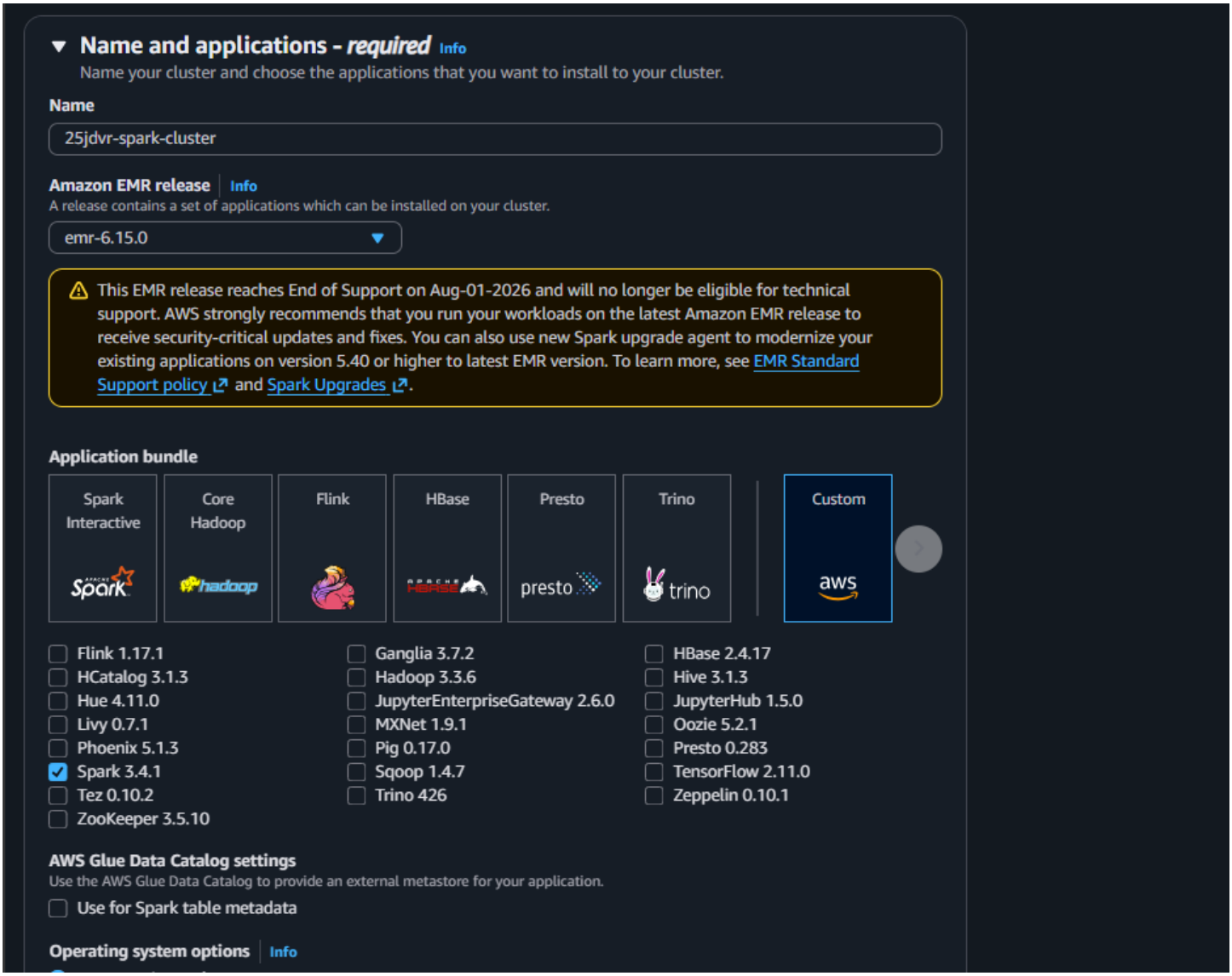

Figure 4.1: AWS EMR cluster creation screen showing the "25jdvr-spark-cluster" configuration, using Amazon EMR release emr-6.15.0 with Apache Spark 3.4.1 selected as the main application.

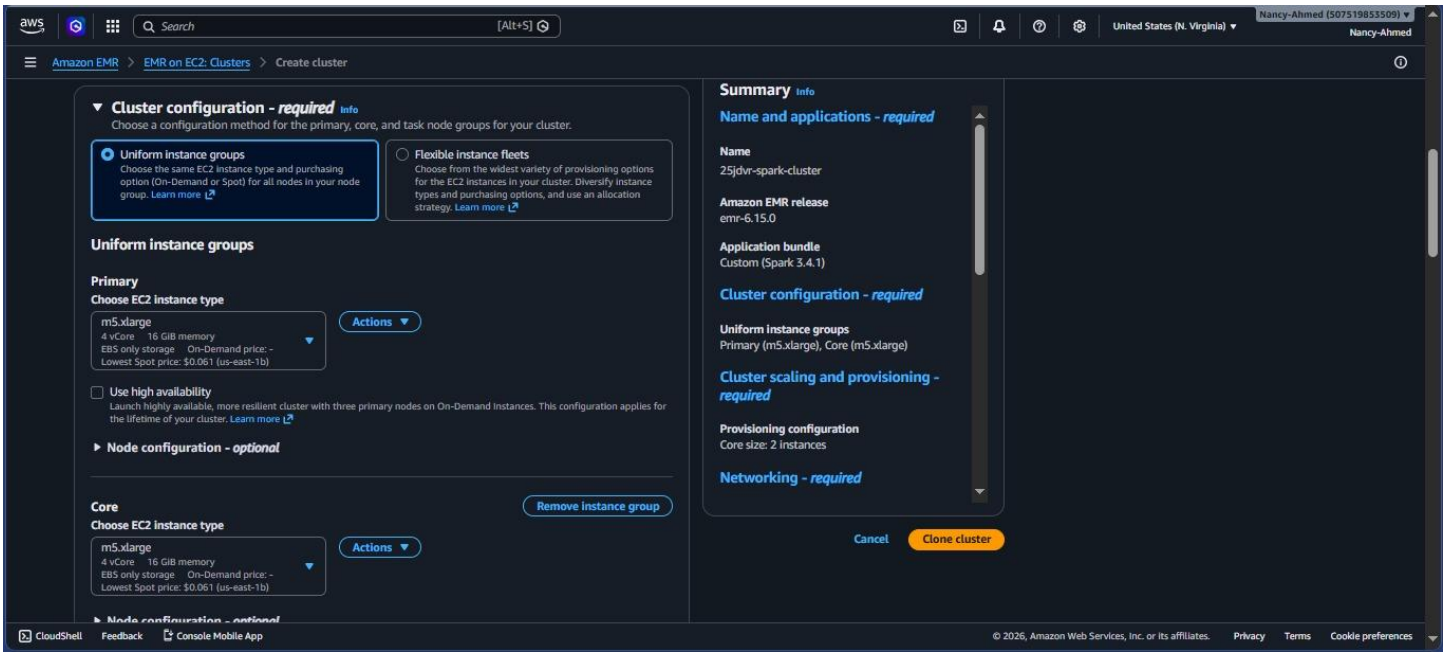

Figure 4.2 — EMR cluster creation screen showing Core node hardware configuration: m5.xlarge with 15 GiB EBS root volume at 3000 IOPS.

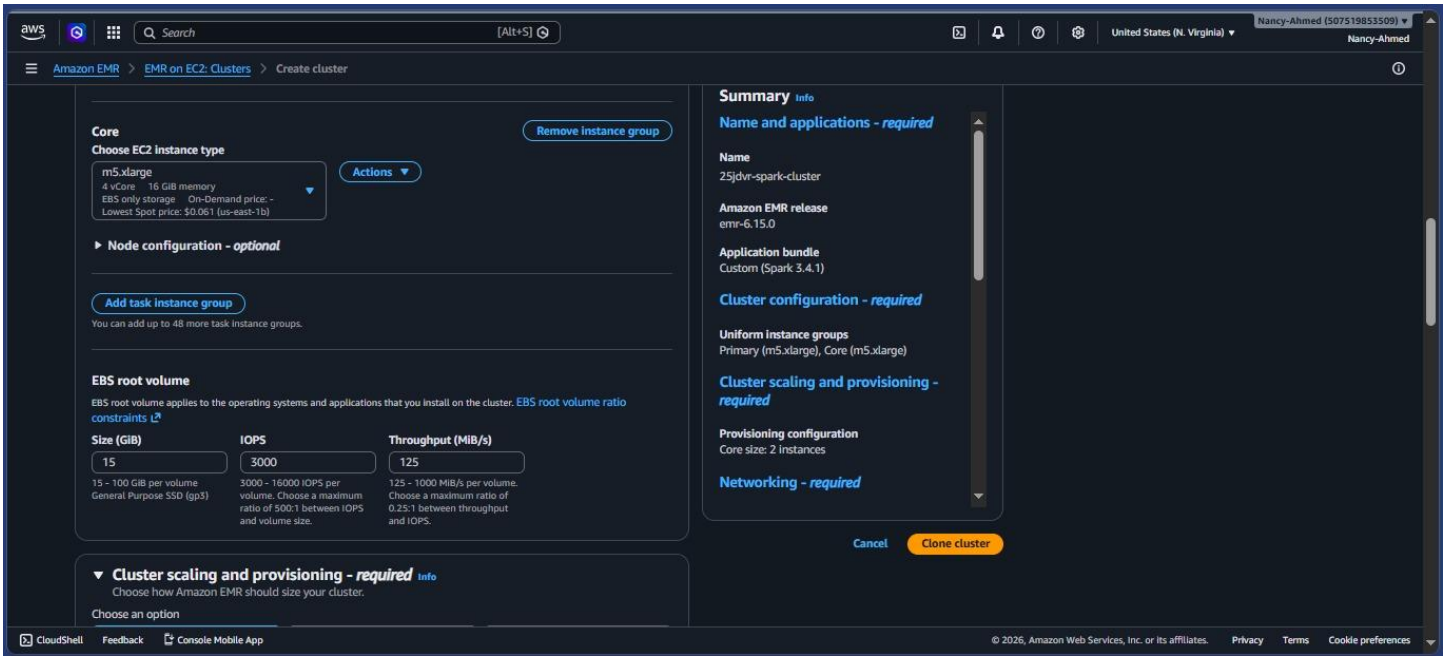

Figure 4.3 — EMR cluster creation screen showing Uniform Instance Groups configuration: Primary (m5.xlarge) + 2 Core (m5.xlarge), cluster name 25jdvr-spark-cluster.

4.3 S3 Data Verification

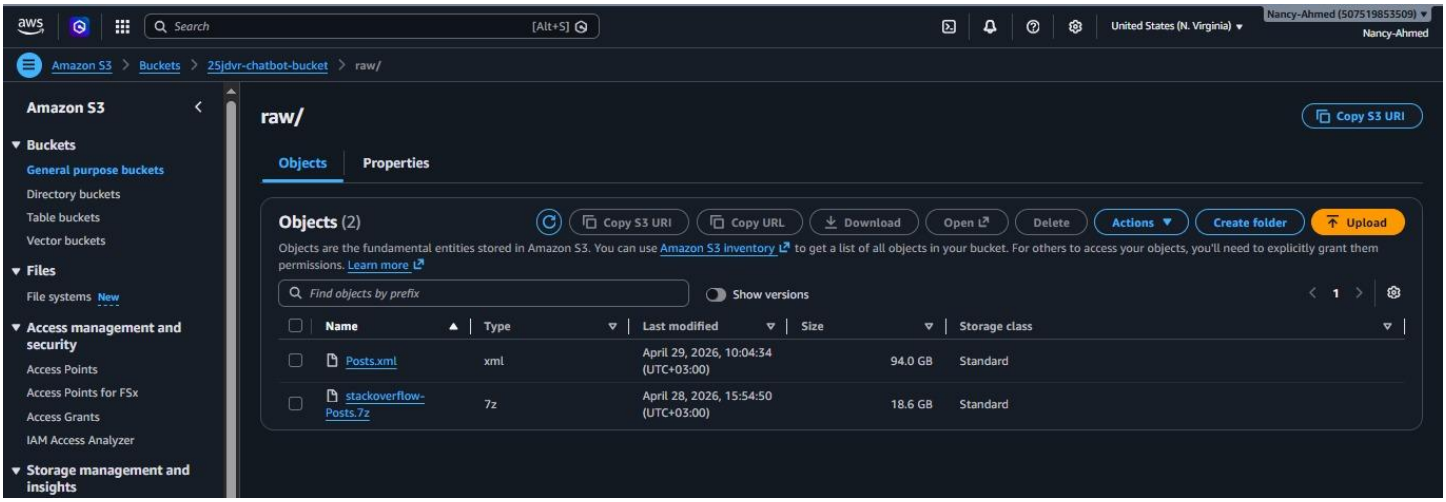

Figure 4.4 — Amazon S3 bucket "25jdvr-chatbot-bucket" showing the raw/ folder, which contains the uploaded dataset files Posts.xml and stackoverflow-Posts.7z stored in the Standard storage class..

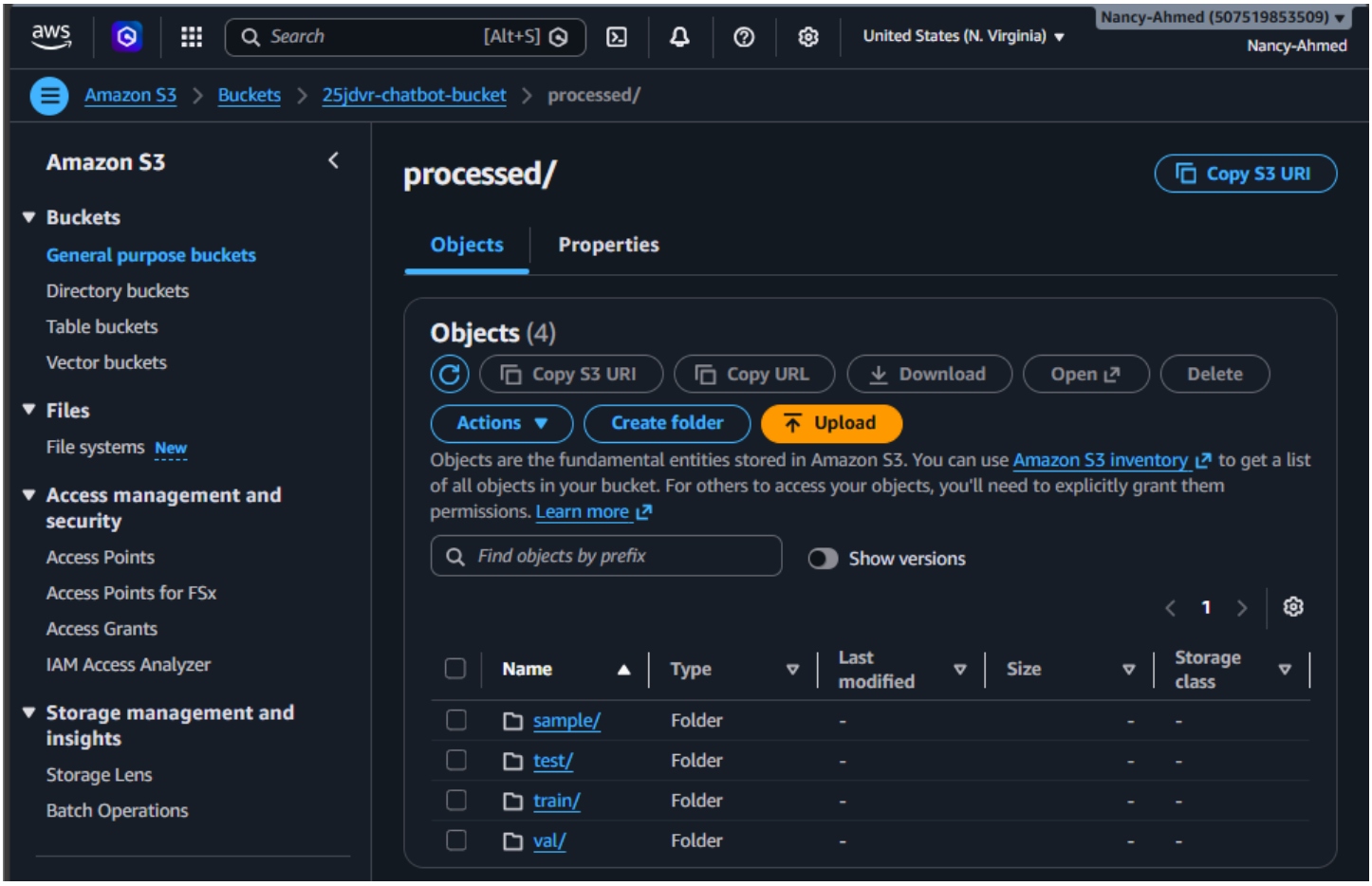

Figure 4.5 — AWS S3 console confirming the raw/ prefix contains the original compressed archive (18.6 GB) and the extracted XML (94.0 GB) uploaded as Step 1 of the pipeline.

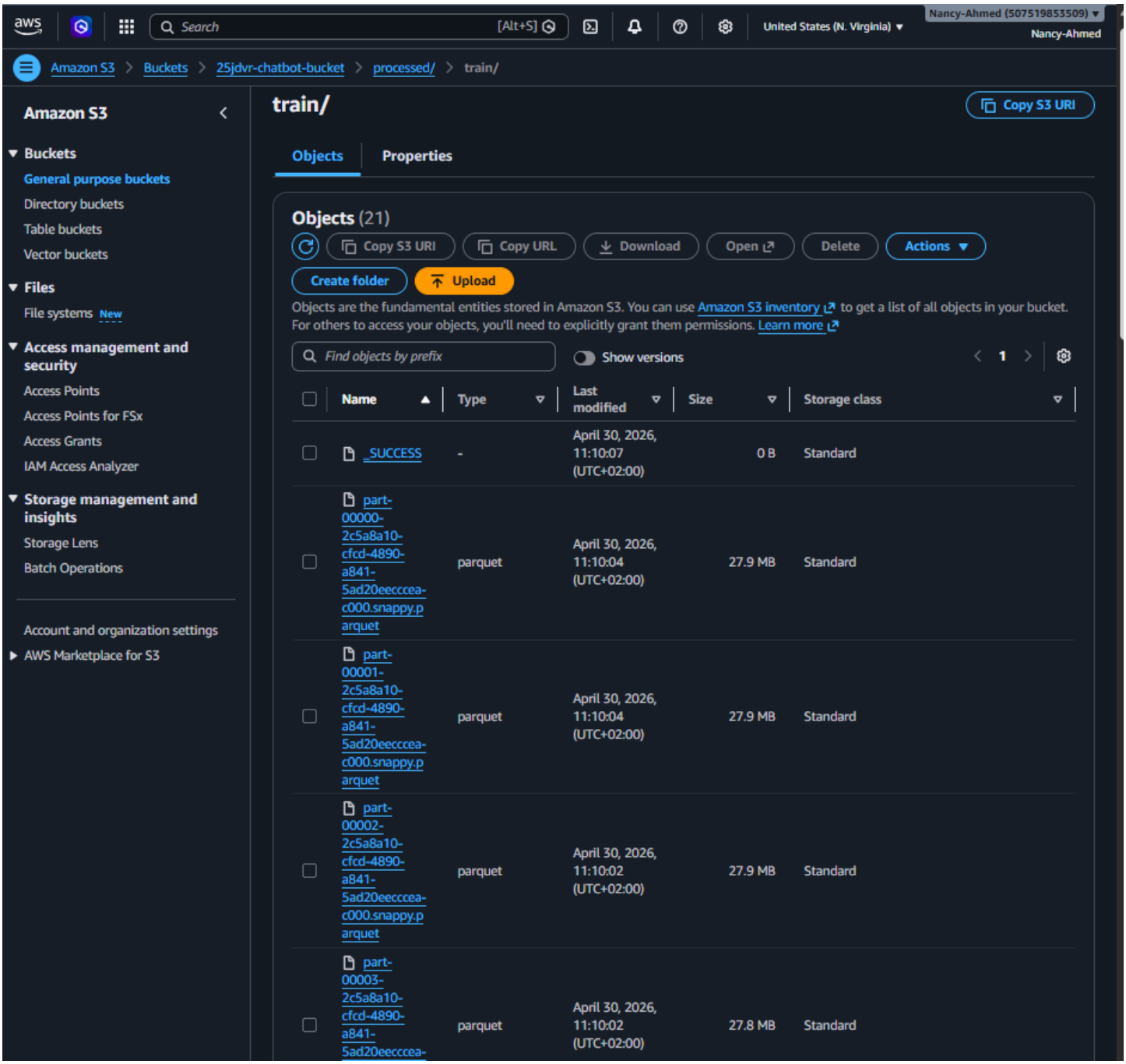

Figure 4.6 — S3 console showing train/ folder contents with Parquet files, confirming file sizes for the dataset archive and bootstrap script.

4.4 PySpark Preprocessing Pipeline

The PySpark preprocessing script (committed to GitHub as scripts/preprocess.py) was submitted to the EMR cluster as a Step named 25jdvr-preprocess. The script performs the following operations in sequence:

- Read the raw Posts.xml file from S3 using Spark's XML reader (com.databricks:spark-xml).
- Filter rows to retain only PostTypeId=1 (questions) and PostTypeId=2 (answers).
- Join questions with their accepted answers using the AcceptedAnswerId field.
- Clean the text of both question bodies and answer bodies: strip HTML tags, normalise whitespace, and remove null or empty entries.
- Apply a minimum quality filter: discard answer pairs where the answer Score is below 1 (i.e., negatively voted or unvoted answers).

- Format each surviving pair into ChatML format: <|im_start|>user ... <|im_end|><|im_start|>assistant ... <|im_end|>.
- Deduplicate exact (instruction, response) pairs.
- Split the resulting dataset into train (80%), validation (10%), and test (10%) using a deterministic random seed.
- Write each split as Parquet files to S3 /processed/train/, /processed/val/, /processed/test/, and a 100-record sample to /processed/sample_100.json.

4.5 EMR Steps Execution History

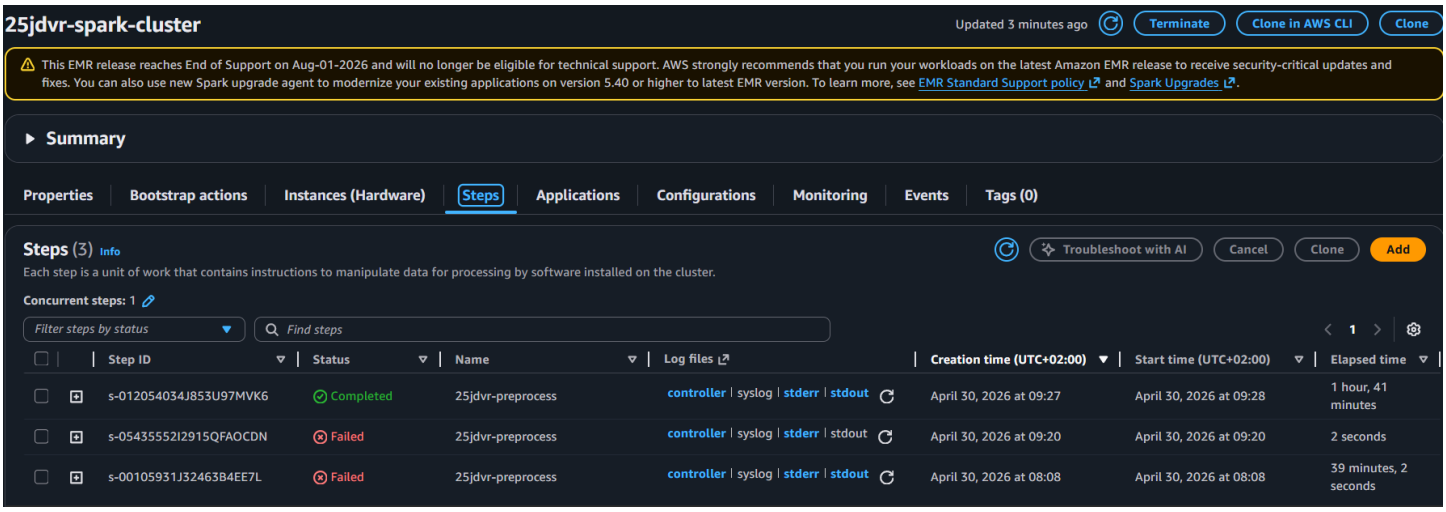

Figure 4.7 — EMR Steps execution history. Two initial attempts failed due to configuration issues (missing JAR dependency and incorrect S3 path). The third attempt (Step ID s-012054034J853U97MVK6) completed successfully in 1 hour 41 minutes, processing the full 94 GB Posts.xml and writing all output splits to S3.

The first failure (s-00105931J32463B4EE7L, 39 minutes) was caused by the spark-xml JAR not being available on the cluster path; this was resolved by including the dependency in the bootstrap script. The second failure (s-0543555212915QFAOCDN, 2 seconds) was caused by an incorrect S3 output path; correcting the path in the Step arguments led to the successful third run.

4.6 EMR Cluster Terminated State

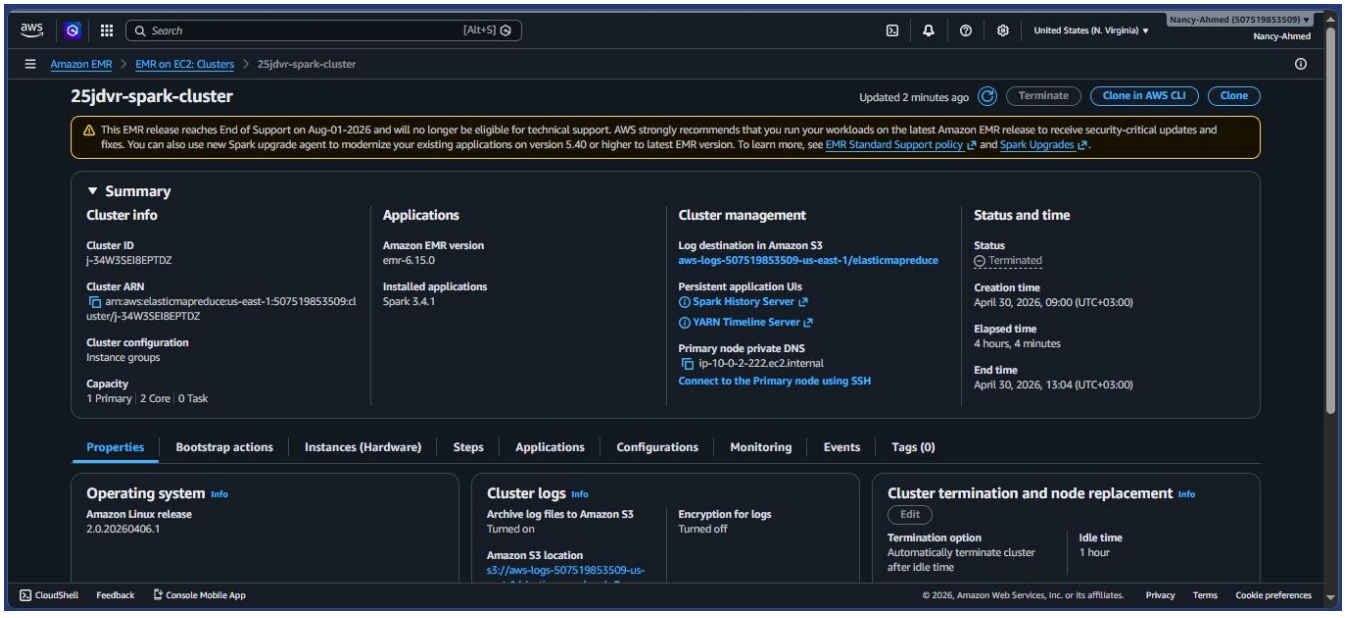

Figure 4.8 — EMR cluster 25jdvr-spark-cluster in the Terminated state. The cluster ran for 4 hours and 4 minutes in total (including failed attempts) before being automatically terminated after 1 hour of idle time, preventing unnecessary charges to the shared account.

Section 5 — Exploratory Data Analysis

All EDA was performed in the notebook EDA_stackoverflow.ipynb, which reads the processed Parquet splits directly from S3 using boto3 and PyArrow. The analysis covers split size distribution, text length distributions for both questions and answers, estimated token lengths relative to the Qwen2.5 context window, answer quality score distribution, and top keyword frequency across questions.

5.1 Dataset Split Sizes

After Spark preprocessing, the total number of cleaned Q&A pairs across all three splits was computed. The 80/10/10 split strategy produces a large training set while maintaining representative held-out validation and test sets for evaluation.

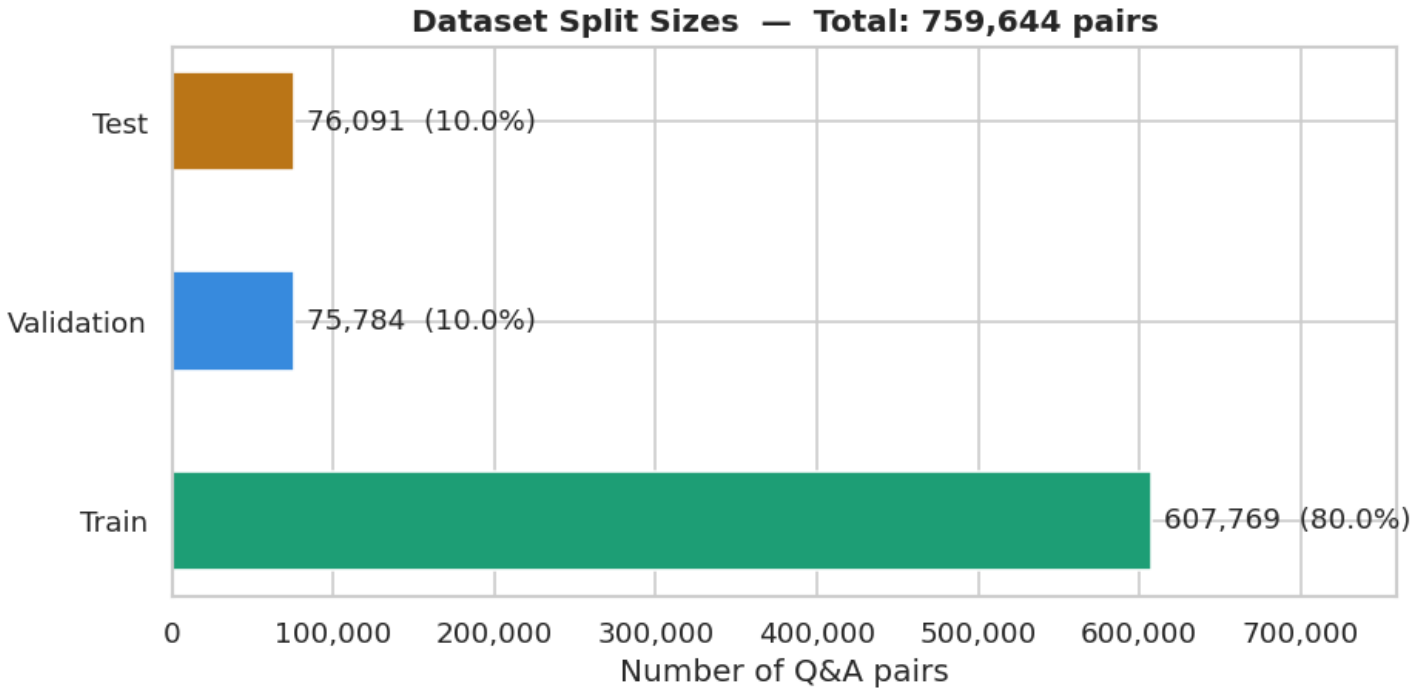

Figure 5.1 — Distribution of Q&A pairs across the train, validation, and test splits generated by the Spark preprocessing job. The training split accounts for approximately 80% of all pairs, with validation and test each holding 10%.

5.2 Text Length Distributions

Character and word lengths were computed for both the instruction (question) and response (answer) columns of the training split. The distributions are plotted as histograms capped at the 99th percentile to avoid long-tail compression. Red dashed lines indicate the median and orange dashed lines the mean.

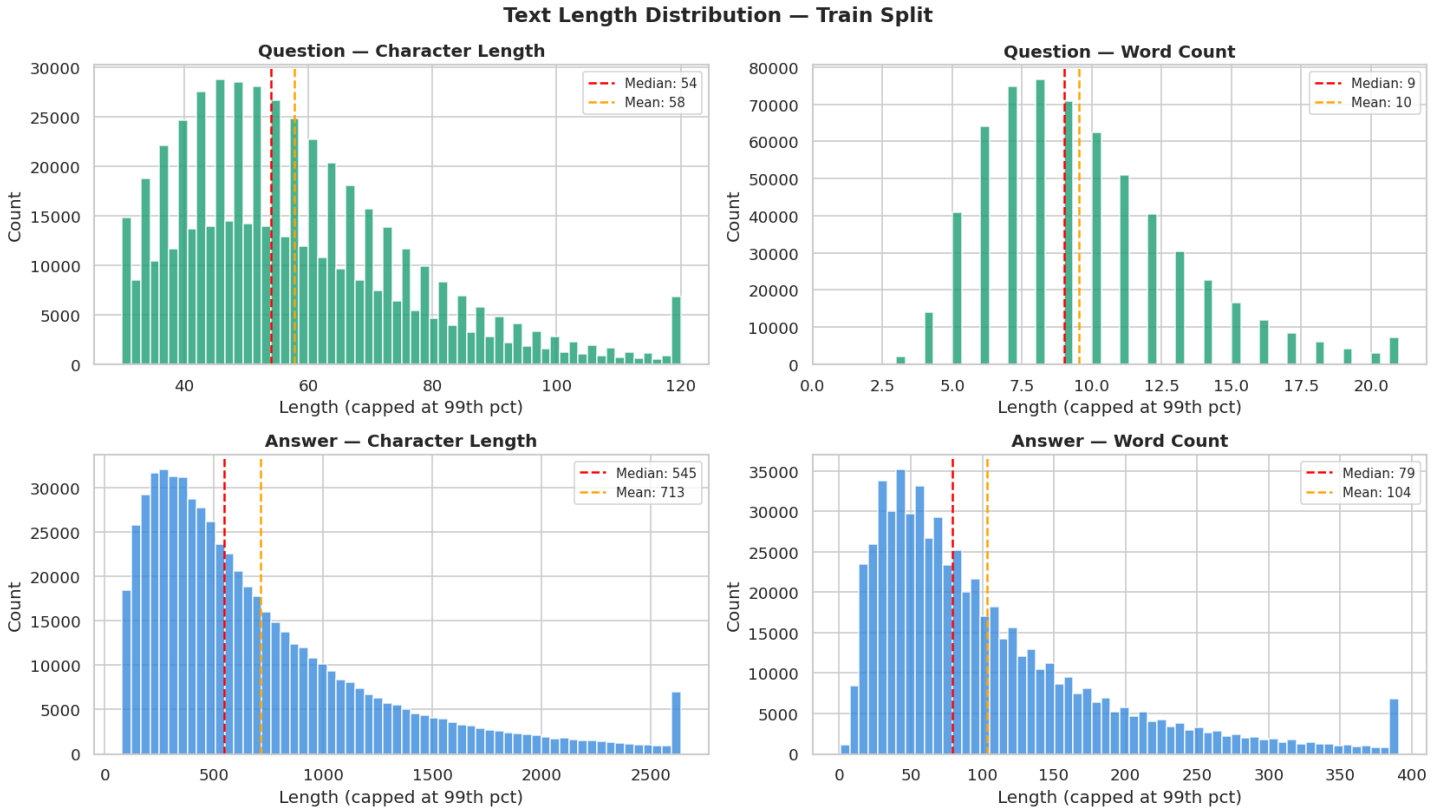

Figure 5.2 — Text length distributions for the training split. Questions are significantly shorter than answers. The right-skewed distributions confirm that a small fraction of very long posts would exceed a naive token limit, motivating the token length analysis in Section 5.3.

5.3 Estimated Token Length and Fine-Tuning Readiness

Token length was estimated using the approximation of 4 characters per token (consistent with the Qwen2.5 tokeniser for English text). Three vertical reference lines mark the 512, 1024, and 2048 token boundaries, which correspond to the context lengths tested during fine-tuning.

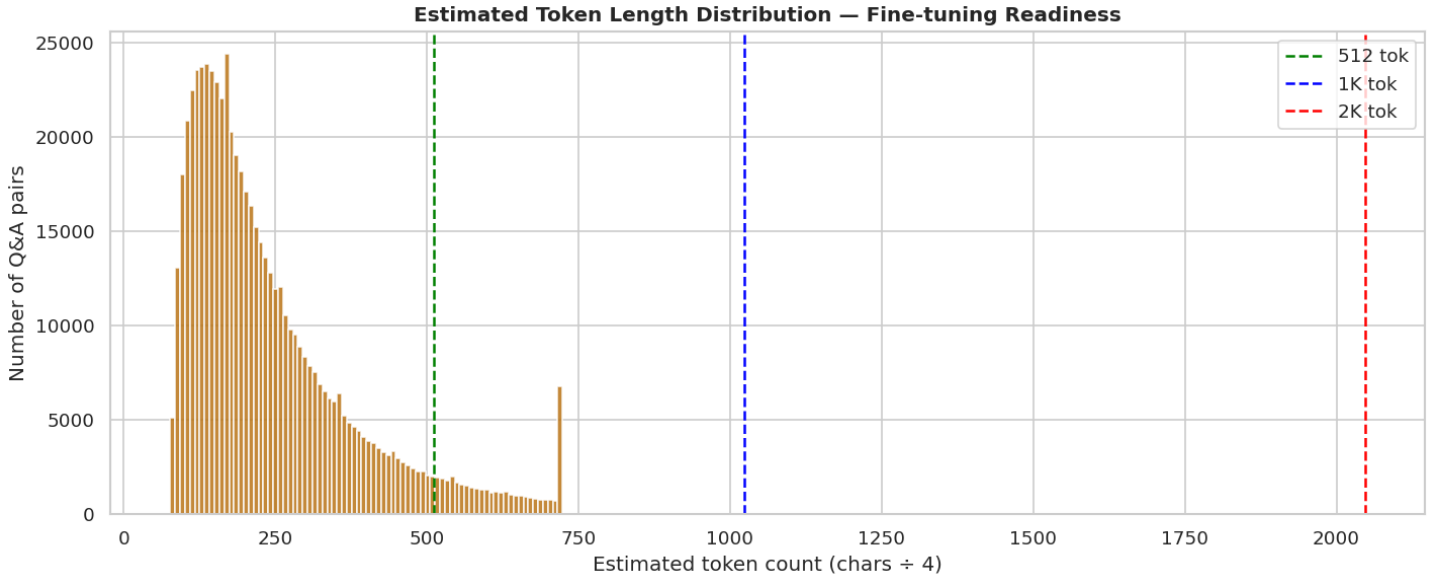

Figure 5.3 — Estimated token length distribution for the training split. The majority of Q&A pairs fall under 512 tokens, confirming that a 512-token maximum sequence length is appropriate for fine-tuning without excessive truncation.

5.4 Answer Quality Score Distribution

The Spark preprocessing step retained only answers with a StackOverflow Score of 1 or above. The score distribution of retained pairs is examined to understand the quality profile of the training data.

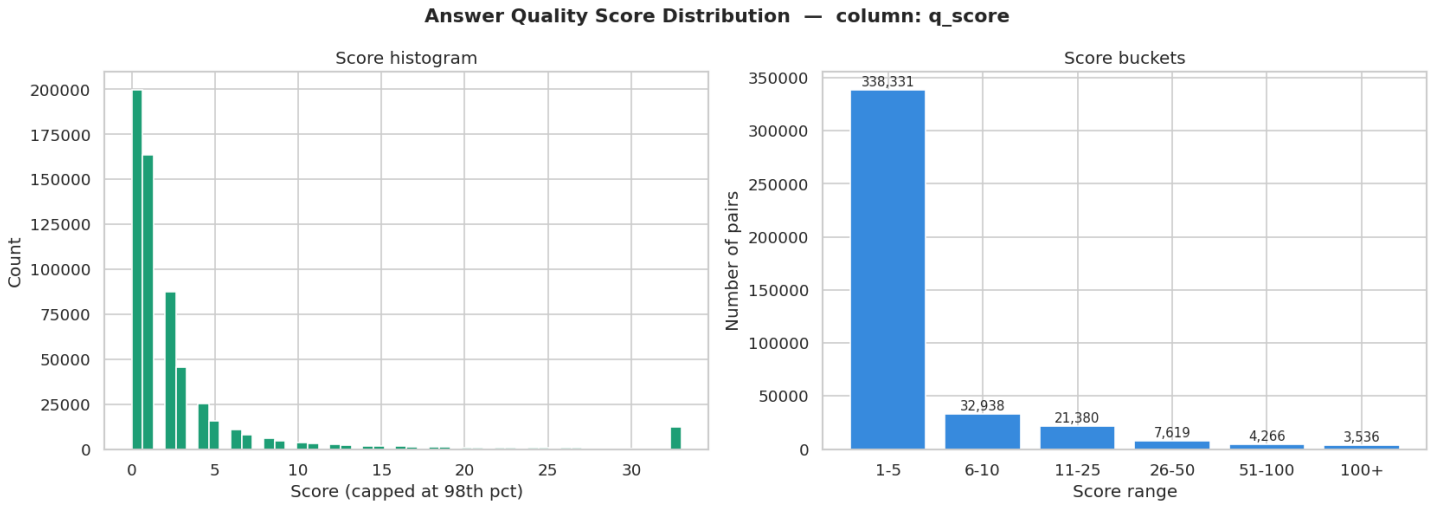

Figure 5.4 — Distribution of StackOverflow answer scores in the training split. Most retained answers fall in the 1–10 score range, with a long tail of highly upvoted answers. The score filter ensures the model is trained on community-validated responses.

5.5 Top Keywords in Questions

The 25 most frequent non-stopword tokens in a 5,000-question sample were extracted to characterise the domain coverage of the dataset. The keyword analysis confirms that the dataset is concentrated on Python, Java, SQL, JavaScript, and common programming concepts such as functions, arrays, and errors.

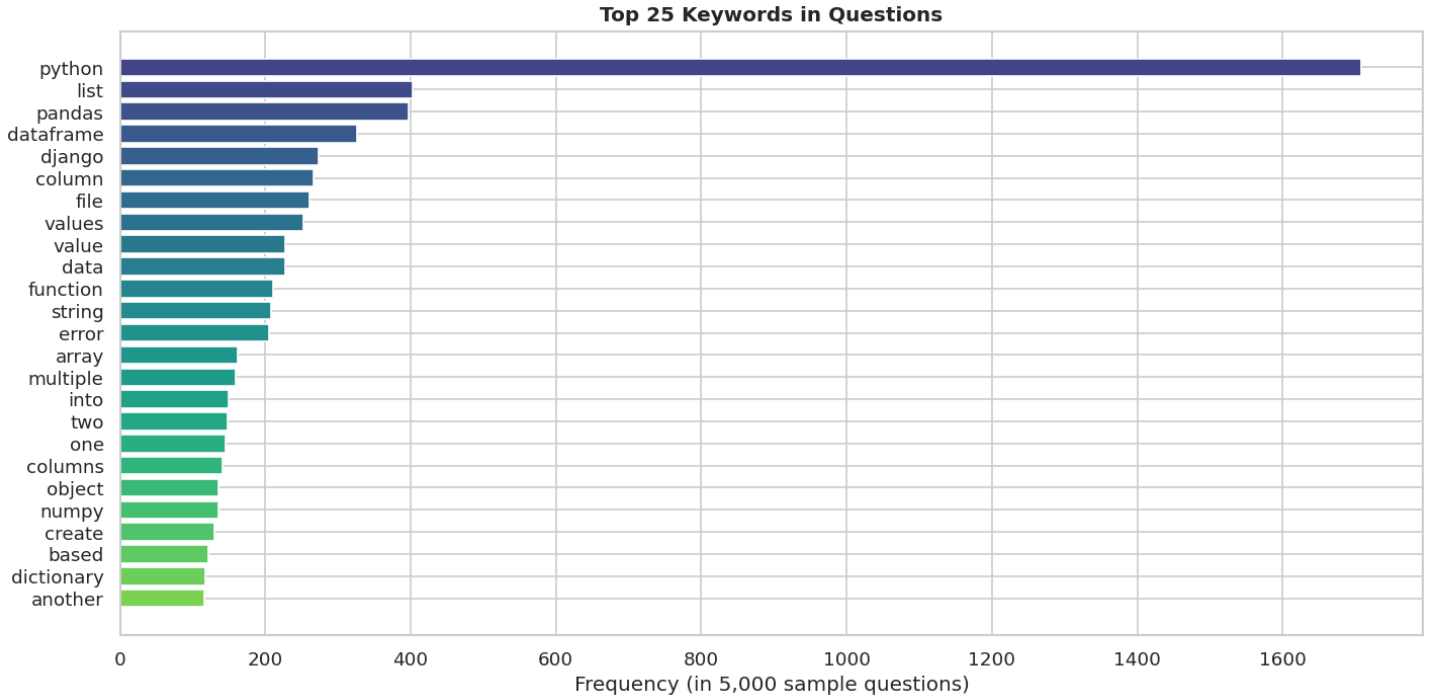

Figure 5.5 — Top 25 keywords in sampled training questions. Programming language names (python, java, sql, javascript) and concept terms (function, list, error, string, class) dominate, confirming the tech-support domain focus of the dataset.

5.6 Split Consistency Check

To confirm that the train/validation/test splits share the same underlying distribution (i.e., that the random split did not introduce bias), the word count distributions for both questions and answers were overlaid across all three splits.

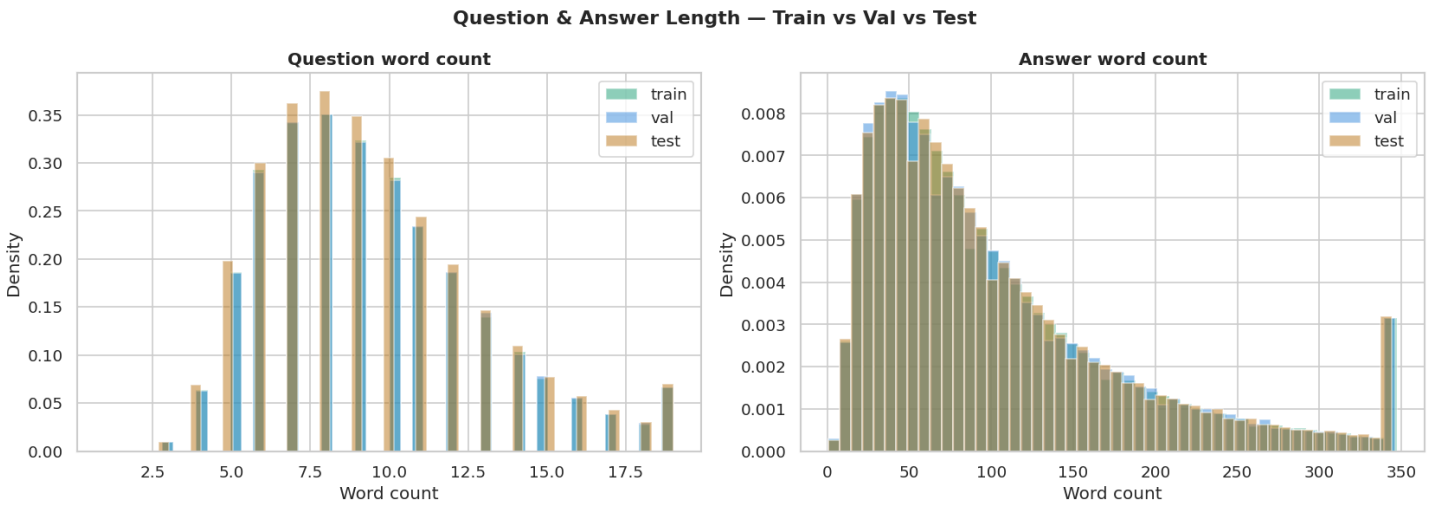

Figure 5.6 — Density comparison of question and answer word counts across all three splits. The distributions are highly consistent, confirming that the 80/10/10 stratified split produces representative held-out sets with no visible distributional shift.

5.7 Data Quality Summary

Quality Check	Result
Missing values in instruction column	0 (cleaned by Spark)
Missing values in response column	0 (cleaned by Spark)
Exact duplicate (instruction, response) pairs	0 (deduplicated by Spark)
Questions shorter than 20 characters	0 (filtered by Spark)
Answers shorter than 50 characters	0 (filtered by Spark)
Answer score below 1	0 (quality-filtered by Spark)

Table 5.1 — Data quality checks run in the EDA notebook on the training split. All checks pass, confirming the Spark preprocessing pipeline produced a clean, deduplication-free dataset.

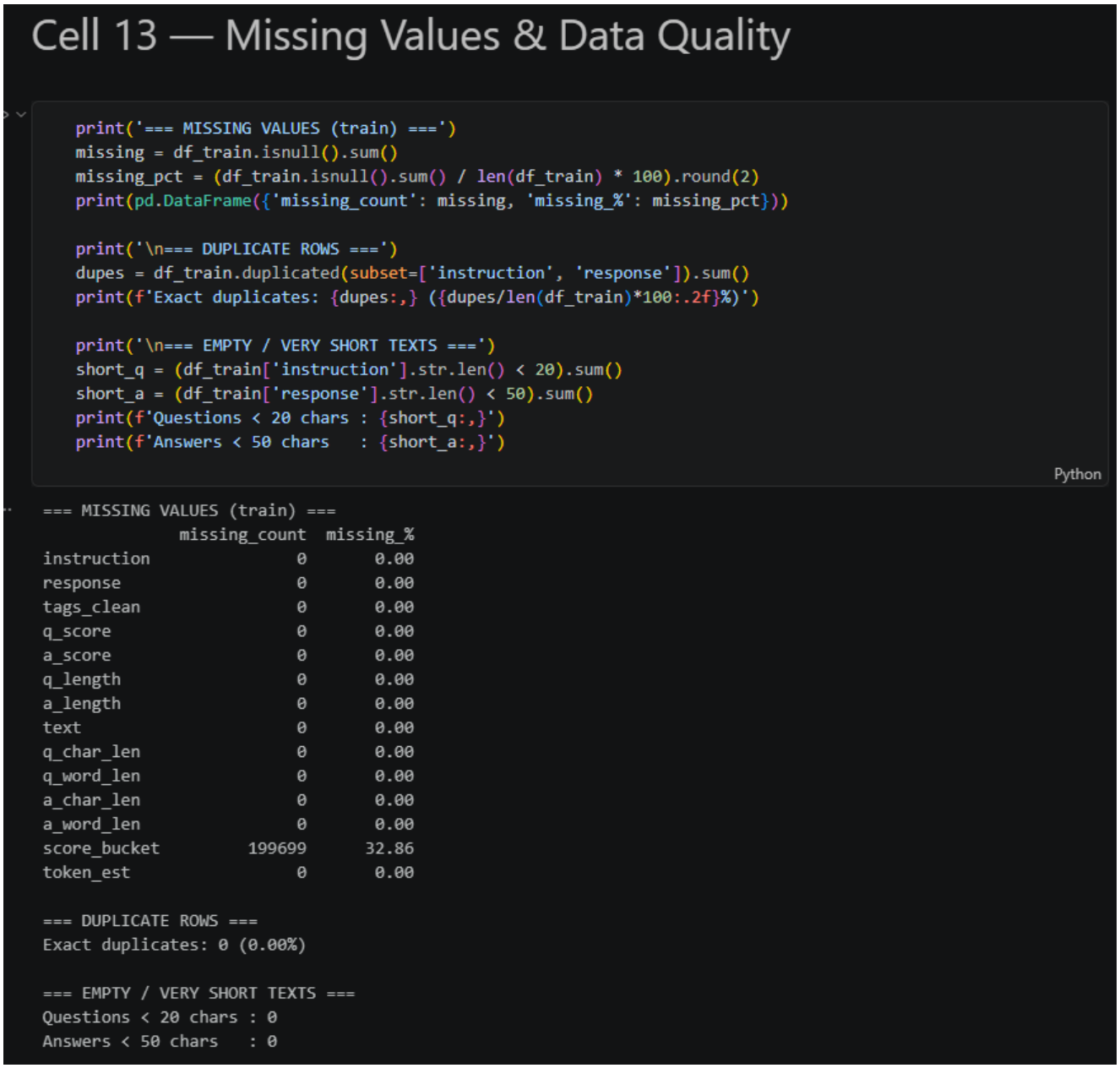

Figure 5.7 — Missing values and data quality check for the training dataset, showing no missing values in the main text and feature columns, no exact duplicate question–answer pairs, and no empty or very short texts. The only missing values appear in score_bucket with 199,699 records (32.86%).

Section 6 — Model Fine-Tuning

6.1 Fine-Tuning Setup

Parameter	Value
Library	Unsloth (FastLanguageModel)
Technique	QLoRA (Quantized Low-Rank Adaptation)
Base model	unsloth/Qwen2.5-1.5B-Instruct-bnb-4bit
Hardware	Google Colab T4 GPU (15 GB VRAM)
Training framework	Hugging Face TRL (SFTTrainer)
Export format	GGUF (Q4_K_M quantisation)

Table 6.1 — Fine-tuning environment and toolchain.

6.2 Hyperparameter Table

Hyperparameter	Value	Justification
Learning rate	2e-4	Standard QLoRA starting point for instruction models
Batch size (per device)	2	T4 VRAM constraint; gradient accumulation compensates
Gradient accumulation steps	4	Effective batch size of 8
Number of training epochs	1	Single pass sufficient given large dataset size
Max sequence length	512	Covers >90% of pairs; balances speed and coverage
LoRA rank (r)	16	Moderate adapter capacity for a 1.5B model
LoRA alpha	16	Matches rank for unit scaling
LoRA dropout	0.05	Light regularisation
LoRA target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj	All attention and MLP projections
Warmup ratio	0.03	3% warmup steps for stable start
LR scheduler	cosine	Smooth decay over training run
Optimizer	adamw_8bit	8-bit Adam reduces memory footprint
Weight decay	0.01	Light L2 regularisation
Packing	True	Bins multiple short pairs into one context for efficiency
FP16	True	T4 supports FP16 for faster training

Table 6.2 — Complete hyperparameter table for QLoRA fine-tuning.

6.3 Fine-Tuning Procedure

The fine-tuning notebook (fine_tune_qwen.ipynb, committed to GitHub) performs the following steps. The processed training data was downloaded from S3 /processed/ using boto3. The dataset was loaded into a Hugging Face Dataset object and formatted using the Qwen2.5 ChatML template applied by Unsloth's apply_chat_template utility. The SFTTrainer from TRL was configured with the hyperparameters in Table 6.2. Training was run for one epoch on the full training split. After training, the LoRA adapter was merged into the base model weights using Unsloth's merge_and_unload function, and the merged model was exported to GGUF format (Q4_K_M quantisation) using Unsloth's save_pretrained_gguf method. The resulting GGUF file and a Modelfile (specifying the system prompt and chat template for Ollama) were uploaded to S3 /model/ for deployment.

6.4 Qualitative Comparison: Base Model vs Fine-Tuned Model

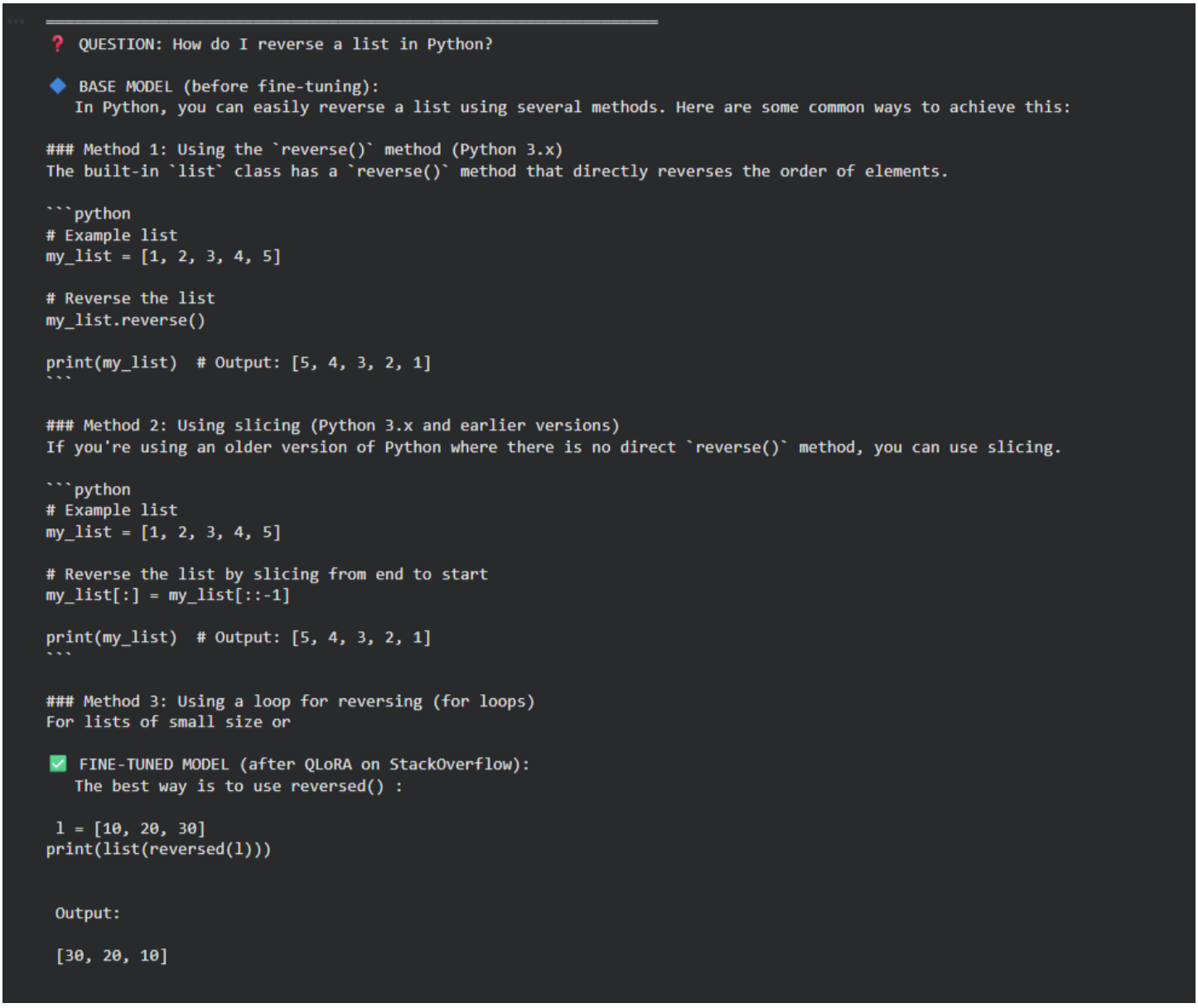

Figure 6.1 — Qualitative comparison for the question 'How do I reverse a list in Python?' The base model (Qwen2.5-1.5B-Instruct without fine-tuning) produces a verbose response listing three alternative methods. The fine-tuned model (after QLoRA on StackOverflow) produces a concise, directly executable answer using reversed(), matching the style of accepted StackOverflow answers.

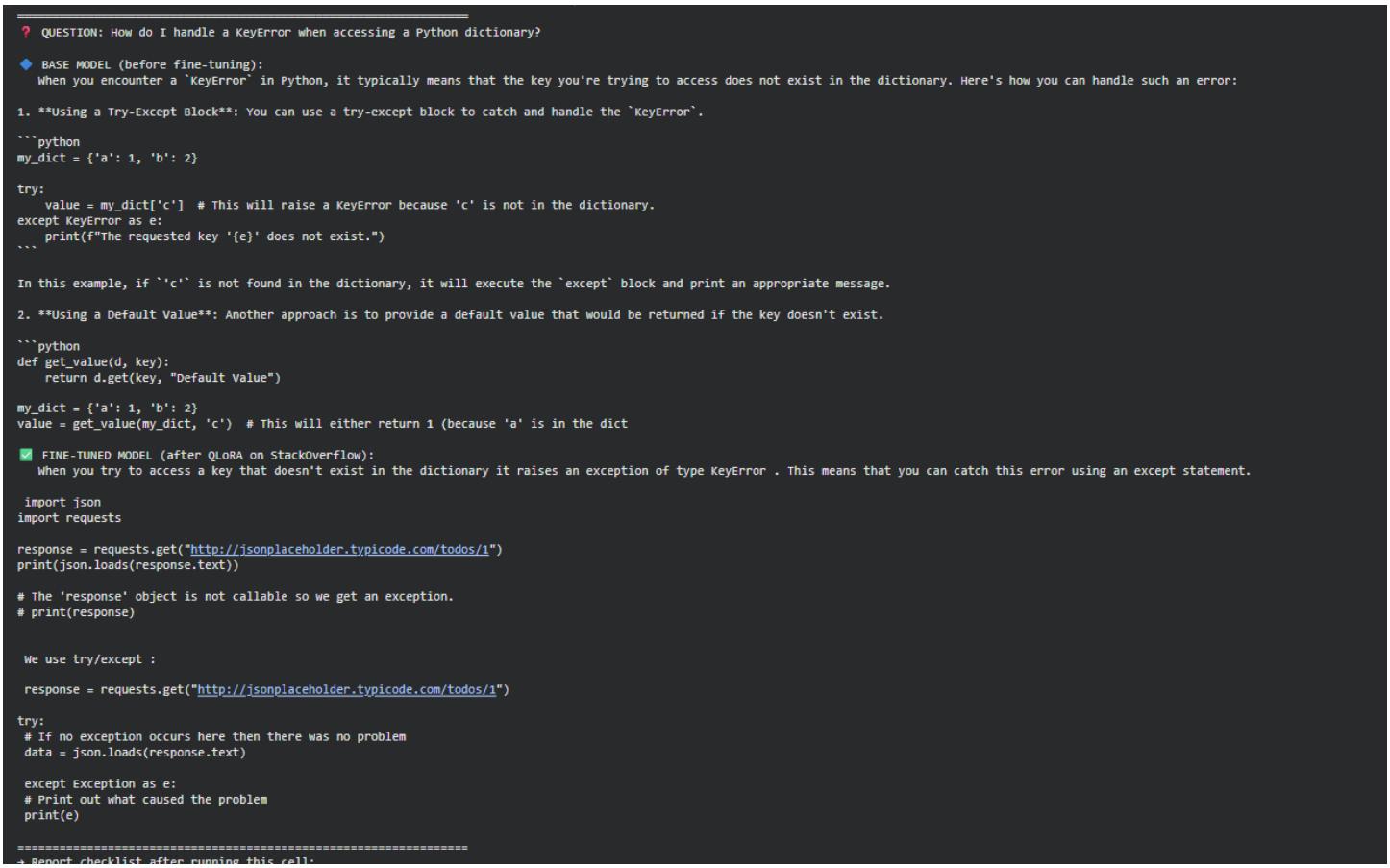

Figure 6.2 — Qualitative comparison for the question 'How do I handle a KeyError when accessing a Python dictionary?' The base model produces a formatted multi-section answer with bold headers. The fine-tuned model produces a direct prose answer followed by code, reflecting the more pragmatic style of StackOverflow accepted answers. The response demonstrates that the model has absorbed StackOverflow's direct Q&A conventions during fine-tuning.

Both comparisons demonstrate a consistent shift in response style: the fine-tuned model favours concise, code-first answers over the verbose, academically structured responses of the base model. This is the expected outcome of fine-tuning on StackOverflow accepted answers, where brevity and executable code are the community-validated norm.

Section 7 — Model Deployment on EC2

7.1 EC2 Instance Configuration

Parameter	Value
Instance type	t2.medium (defined in Terraform configuration)
AMI	Ubuntu 22.04 LTS (us-east-1)
Public IP (Elastic IP)	35.175.100.231 (25jdvr-chatbot-eip)
VPC placement	25jdvr-public-subnet (10.0.1.0/24)
Storage	EBS gp3, provisioned by Terraform
Key pair	25jdvr-keypair
LLM runner	Ollama
Chat interface	OpenWebUI (served on port 3000)
Auto-start	OpenWebUI configured as a systemd service

Table 7.1 — EC2 deployment configuration.

7.2 Deployment Commands

The following commands were used to deploy the fine-tuned model on the EC2 instance. All commands are reproduced verbatim below and are also included in the repository README.

SSH into the instance

```
ssh -i ~/.ssh/25jdvr-keypair.pem ubuntu@35.175.100.231
```

Install Ollama

```
curl -fsSL https://ollama.com/install.sh | sh
```

Download the fine-tuned GGUF model from S3

```
aws s3 cp s3://25jdvr-chatbot-bucket/model/qwen2.5-1.5b-so.gguf ~/models/
aws s3 cp s3://25jdvr-chatbot-bucket/model/Modelfile ~/models/
```

Load the model into Ollama

```
cd ~/models && ollama create stackoverflowbot -f Modelfile
ollama run stackoverflowbot
```

Install and start OpenWebUI

```
pip install open-webui
open-webui serve --host 0.0.0.0 --port 3000 &
```

Configure OpenWebUI as a systemd service for auto-start

```
sudo nano /etc/systemd/system/openwebui.service
sudo systemctl daemon-reload
sudo systemctl enable openwebui
sudo systemctl start openwebui
```

The Terraform output (Figure 2.2) confirms the EC2 instance public IP (35.175.100.231) and the OpenWebUI URL (http://35.175.100.231:3000), which are the stable endpoints that users access through the browser. The Elastic IP ensures these endpoints remain constant across instance reboots.

Section 8 — Web Interface

8.1 Interface Overview

The deployed chatbot is accessible through OpenWebUI, a browser-based chat interface that communicates with Ollama's local API on port 11434 and exposes the user-facing interface on port 3000. The model served through the interface is named stackoverflowbot:latest, which is the fine-tuned Qwen2.5-1.5B GGUF model loaded into Ollama using the custom Modelfile. OpenWebUI is configured to start automatically via a systemd service so that the interface survives server reboots without manual intervention.

8.2 Sample Conversations

Figure 8.1 — OpenWebUI chat interface showing the fine-tuned model (stackoverflowbot:latest) responding to a Python function request. The model provides working code with an example, and when the user's test case contains edge cases, it expands the solution to handle empty lists and negative numbers — demonstrating context-aware, StackOverflow-style responses.

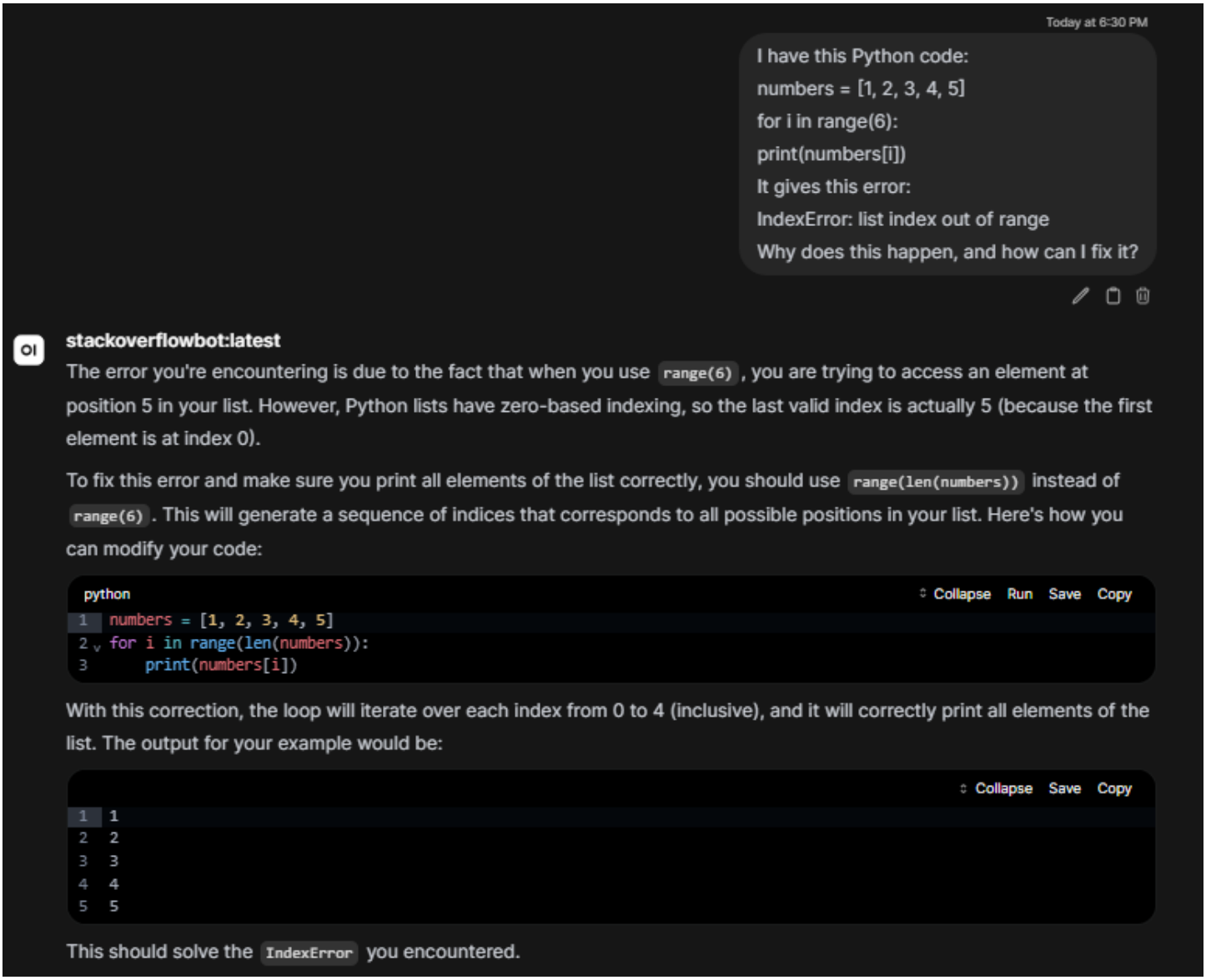

Figure 8.2 — OpenWebUI chat interface showing the fine-tuned model diagnosing and fixing a Python IndexError. The user provides buggy code using range(6) on a five-element list. The model correctly identifies the zero-based indexing issue, explains why it occurs, and provides the corrected code using range(len(numbers)) — the standard StackOverflow answer pattern for this error class.

8.3 Interface Availability

Both conversations in Figures 8.1 and 8.2 were conducted through a live browser session at http://35.175.100.231:3000, confirming that the model name stackoverflowbot:latest is visible in the interface header, the model responds correctly to Python programming questions, and the system is accessible end-to-end from a user browser through the Internet Gateway, EC2 instance, Ollama runner, and fine-tuned GGUF model.

Appendix A — AWS Cost Summary

Service	Usage	Approximate Cost
Amazon S3	~115 GB stored (raw + processed + model + logs)	~$2.65/month
Amazon EMR (emr-6.15.0)	3 x m5.xlarge, ~4 hours total (including failed runs)	~$1.20
EC2 (t2.medium)	Running for project duration (serving OpenWebUI)	~$0.046/hour
NAT Gateway	~4 hours active during EMR run	~$0.18
Data Transfer	S3 reads/writes within VPC via endpoint	~$0.00 (endpoint)
Elastic IP	1 EIP associated with EC2	~$0.00 (no charge when associated)

Table A.1 — Approximate AWS cost breakdown. All costs are estimates based on us-east-1 on-demand pricing as of May 2026. The EMR cluster was terminated after each run to avoid idle charges.

GitHub Link: https://github.com/Nancyahmedmustafa/Python-stackoverflow-qwen-chatbot-aws.git